CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 08
Greedy Algorithms, Exchange Arguments & Matroids

Greedy algorithms are easy to write and easy to get wrong. Today is about the proofs, and about the structure that makes greedy work.

Saturday 14 November 2026 · 3 hours

Midterm (L01–L07) today in a separate exam slot · Project M3 due today

Motivating problem: which greedy rule?

Jobs with processing times t_j and deadlines d_j, one machine, no idle time needed. Lateness $\ell_j = \max(0, C_j - d_j)$. Minimise the **maximum lateness** $L = \max_j \ell_j$.

Three plausible rules:

1. Shortest processing time first
2. Smallest slack $d_j - t_j$ first
3. Earliest deadline first (EDF)

PAIRS · 5 MIN

Two of the three rules are wrong. Find a counterexample with **two jobs** for each wrong rule. Then: what would it take to *prove* the third one?

BLOCK A · 00:10–01:00

Proving greedy algorithms correct

The template · staying ahead · exchange · counterexamples

The greedy template

A greedy algorithm makes one irrevocable choice at a time, based on a local rule. To prove it optimal, show:

1. **Greedy choice is safe:** some optimal solution contains the first greedy choice.
2. **Optimal substructure:** after fixing that choice, the rest of an optimal solution is optimal for the remaining subproblem, which has the same form.
3. **Induction** on the size of the problem glues these together.

Two common proof styles:

- **Greedy stays ahead:** by induction, greedy's partial solution is at least as good as any other solution's after each step.
- **Exchange:** transform any optimal solution step by step into greedy's without making it worse.

WHAT IS NOT A PROOF

- "It takes the best option at each step"
- "It worked on all my test cases"
- "It is intuitive"

Every rule in the motivating problem "takes the best option" by *its own* measure.

Greedy stays ahead: interval scheduling

Greedy (earliest finish) picks $g_1, \ldots, g_k$; any optimal solution picks $o_1, \ldots, o_m$. Both lists are sorted by finish time.

LEMMA (STAYS AHEAD)

For all $r \le k$: $f(g_r) \le f(o_r)$.

PROOF

$r = 1$: greedy picks the earliest finish overall. Step: o_r starts after o_{r-1} finishes, and $f(o_{r-1}) \ge f(g_{r-1})$ by the hypothesis. So o_r is compatible with $g_1..g_{r-1}$ and was available to greedy, which picked the earliest-finishing available interval: $f(g_r) \le f(o_r)$.

■

OPTIMALITY

Suppose $m > k$. Then o_{k+1} starts after $f(o_k) \ge f(g_k)$, so it is compatible with all of greedy's picks, and greedy would not have stopped. So $m \le k$.

■

Exchange argument: EDF minimises maximum lateness

An **inversion**: jobs i, j with $d_i < d_j$ but j scheduled before i. EDF has no inversions (ties in any order) and no idle time.

FACTS

1. All schedules without inversions or idle time have the same L: they differ only in the order of equal-deadline jobs.
2. A schedule with an inversion has an **adjacent** inversion.
3. Swapping an adjacent inversion does not increase L, and it reduces the number of inversions by 1.

PROOF OF (3)

Adjacent j, i with $d_i < d_j$, j first. After the swap, the other jobs are unchanged and i finishes earlier. j now finishes at $C'_j = C_i$ (the old finish time of i), so

$$\ell'_j = C_i - d_j \leq C_i - d_i = \ell_i \leq L.$$

So the new maximum is $\leq L$. ■

Conclusion: take an optimal schedule and remove idle time (this never hurts). Swap adjacent inversions until none remain (finitely many steps, since the inversion count decreases). The result has the same L as EDF and $L \leq$ optimal.

Counterexamples: a library of failures

Problem	Plausible greedy	Smallest counterexample
Interval scheduling (count)	shortest first	$[0,5), [4,7), [6,11)$
Interval scheduling (count)	earliest start	$[0,10), [1,2), [3,4)$
Weighted intervals	earliest finish	$[0,10)$ value 10 vs. $[0,1), [1,2)$ values 1, 1
Coin change	largest coin	$\{1,3,4\}$, amount 6
0/1 knapsack	best value/weight	$W = 10$: $(6,7), (5,5), (5,5)$
Max lateness	shortest job / least slack	(motivating problem)
Set cover	largest set first	not optimal, but within H_n (L14)

> Method: find what the rule **ignores** (length, value, the future's structure) and build an instance where that ignored quantity decides the answer.

Exercise A: prove or refute

1. **Fractional knapsack:** take items in decreasing value/weight order, the last one fractionally. Prove optimality with an exchange argument.
2. **Refuelling:** a car with range R drives along a road with fuel stations at positions $x_1 < \cdots < x_n$ and must reach D. Greedy: refuel only when the next station is out of range, at the farthest reachable station. Prove it minimises the number of stops (staying ahead).
3. **Covering points:** cover points $x_1 < \cdots < x_n$ on a line with the fewest closed intervals of length 1. Greedy: place $[p, p+1]$ at the leftmost uncovered point p, and repeat. Optimal? Prove or refute.
4. **Graph colouring:** colour vertices in order of decreasing degree with the smallest available colour. Optimal?

Pairs · 10 min

BLOCK B · 01:10–02:00

Huffman coding and matroids

A greedy algorithm with a subtle proof · the structure behind greedy

Huffman coding

Symbols with frequencies $f(c)$. A **prefix code** is a binary tree with symbols at the leaves. Its cost is $B(T) = \sum_c f(c)\, d_T(c)$ (the total encoded length).

```
1 HUFFMAN(C)
2   Q = min-priority queue of the symbols, by f
3   repeat |C| − 1 times
4     x = EXTRACT-MIN(Q) ; y = EXTRACT-MIN(Q)
5     z = new node with children x, y
6     f(z) = f(x) + f(y) ; INSERT(Q, z)
7   return the remaining node
```

$O(n \log n)$ with a binary heap. $O(n)$ if the frequencies are already sorted (two queues).

LEMMA 1 (SAFE CHOICE)

If x, y have the two smallest frequencies, some optimal tree has x, y as sibling leaves at maximum depth.

LEMMA 2 (SUBSTRUCTURE)

Replace x, y by a new symbol z with $f(z) = f(x) + f(y)$. If T' is optimal for the new alphabet, then expanding z into x, y gives an optimal T. Here $B(T) = B(T') + f(x) + f(y)$.

Huffman: the proofs

LEMMA 1 (EXCHANGE)

Take any optimal tree T. A full binary tree has two sibling leaves a, b at maximum depth. W.l.o.g. $f(x) \le f(a)$ and $f(y) \le f(b)$. Swap x with a:

$$B(T) - B(T') = \big(f(a) - f(x)\big)\big(d_T(a) - d_T(x)\big) \ge 0,$$

since both factors are ≥ 0. So the swap does not increase the cost. Swap y with b likewise. The result is optimal and has x, y as siblings at maximum depth.

■

LEMMA 2 (REDUCTION)

For any tree T with x, y as siblings, collapsing them into z gives T' with $B(T) = B(T') + f(x) + f(y)$, since x, y are one level deeper than z. If T (from T' optimal) were not optimal, take an optimal T^* with x, y as siblings (Lemma 1) and collapse it: $B(T^{*\prime}) = B(T^*) - f(x) - f(y) < B(T) - f(x) - f(y) = B(T')$, which contradicts the optimality of T'.

■

Theorem (induction on $|C|$): Huffman produces an optimal prefix code.

The MST connection (preview of L09)

Kruskal is greedy: scan the edges by increasing weight and add each edge that does not create a cycle.

CUT PROPERTY

For any cut $(S, V \setminus S)$, a lightest edge crossing the cut belongs to some MST.

EXCHANGE PROOF

Let e be the lightest crossing edge and T an MST without e. $T + e$ has a cycle, which crosses the cut at another edge e', with $w(e') \geq w(e)$. $T + e - e'$ is a spanning tree of weight $\leq w(T)$.

■

A PATTERN

Interval scheduling, EDF, Huffman, MST: each proof exchanges one element of an optimal solution for greedy's choice.

Question: is there a single structure that explains *why* this exchange always works for some problems (MST), and not for others (knapsack)?

Yes: **matroids**.

Matroids

DEFINITION

A matroid $M = (S, \mathcal{I})$: a finite set S and a family $\mathcal{I}$ of *independent* subsets with

1. $\emptyset \in \mathcal{I}$;
2. **hereditary:** $B \in \mathcal{I},\ A \subseteq B \Rightarrow A \in \mathcal{I}$;
3. **exchange:** $A, B \in \mathcal{I},\ |A| < |B| \Rightarrow \exists x \in B \setminus A$: $A \cup \{x\} \in \mathcal{I}$.

A consequence: all maximal independent sets (**bases**) have the same size.

Example	Independent sets
Uniform $U_{k,n}$	subsets of size $\leq k$
Graphic	edge sets that contain no cycle (forests)
Linear	linearly independent sets of vectors
Partition	at most k_i elements from block i
Scheduling (next slides)	sets of unit jobs that can all meet their deadlines
Not a matroid	**knapsack-feasible sets; matchings in general graphs**

The greedy theorem (Rado 1957, Edmonds 1971)

```
GREEDY(S, I, w)
  A = ∅
  for x ∈ S in order of decreasing w(x)      ▷ only positive weights matter
    if A ∪ {x} ∈ I:  A = A ∪ {x}
  return A
```

THEOREM

If $(S, \mathcal{I})$ is a matroid, GREEDY returns a maximum-weight independent set for **every** weight function $w \geq 0$. Conversely, if $(S, \mathcal{I})$ is hereditary and GREEDY is optimal for every w, then $(S, \mathcal{I})$ is a matroid.

PROOF (MATROID ⇒ OPTIMAL)

Let $G = \{g_1, g_2, \dots\}$ (in greedy order) and let $O = \{o_1, o_2, \dots\}$ be optimal, sorted by decreasing weight. We may assume both are bases, so $|G| = |O|$. Claim: $w(g_i) \geq w(o_i)$ for all i. If not, take the first i with $w(g_i) < w(o_i)$. Apply exchange to $A = \{g_1 .. g_{i-1}\}$ and $B = \{o_1 .. o_i\}$: some o_j ($j \leq i$) extends A, with $w(o_j) \geq w(o_i) > w(g_i)$. Greedy would have considered o_j before g_i and taken it (hereditary: it is still addable at that earlier time). A contradiction. So $w(G) \geq w(O)$.

■

A scheduling matroid: unit jobs with deadlines

Unit-time jobs j with deadlines d_j and penalties p_j (paid if late). Minimise the total penalty = **maximise the total penalty of on-time jobs**.

Call a set A of jobs **independent** if all of A can finish by their deadlines.

> **LEMMA**
> A is independent iff for every t: $N_t(A) = |\{j \in A : d_j \leq t\}| \leq t$.

This is a matroid (exchange: if $|B| > |A|$, add the job of $B \setminus A$ with the latest… see the exercise sheet). So **greedy by decreasing penalty**, keeping independence, is optimal. Schedule the chosen jobs in EDF order.

> **WHY IT MATTERS**
> Recognising a matroid gives you, for free: (1) an optimal greedy algorithm, (2) its correctness proof, (3) optimality for *every* weight function (robust to changing penalties).
>
> Checking independence is $O(n)$ per test with counters, so $O(n^2)$ total, or $O(n \log n)$ with DSU on time slots (L09 connection).

BLOCK C · 02:10–02:55

Workshop

Matroid or not? · design with proof · find the gap

Exercise C1: matroid or not?

For each $(S, \mathcal{I})$: is it a matroid? Prove exchange, or give a counterexample to it. If it is a matroid, state what GREEDY computes.

1. S = the edges of a graph; $\mathcal{I}$ = edge sets in which every vertex has degree ≤ 1 (matchings).
2. S = the columns of a matrix over $\mathbb{R}$; $\mathcal{I}$ = linearly independent column sets.
3. S = the edges of a directed graph; $\mathcal{I}$ = edge sets in which every vertex has in-degree ≤ 1.
4. S = items with sizes; $\mathcal{I}$ = sets of total size $\leq W$.
5. S = the edges of a graph; $\mathcal{I}$ = edge sets whose removal leaves the graph connected (for a connected graph).

Groups · 12 min

Exercise C2: find the gap in the proof

Claim. "To minimise the number of coins for amount T with denominations $1 = c_1 < c_2 < \cdots < c_k$, greedy (the largest coin first) is optimal."

> **STUDENT PROOF**
>
> "Let c be the largest coin $\leq T$. Some optimal solution uses c: if an optimal solution O does not use c, its coins smaller than c sum to at least c, so we can replace some of them by one c, which does not increase the count. Then recurse on $T - c$."

1. Find the false step. Give a counterexample to the claim.
2. For which denomination systems is the step true? Prove it for $\{1, 5, 10, 25\}$ or $\{1, 2, 4, 8, \dots\}$.
3. What would the corresponding exchange argument need, and why does it hold for powers of 2?

Groups · 10 min

Synthesis

1. Greedy = safe first choice + optimal substructure + induction. Prove it by **staying ahead** or by **exchange**.
2. Try to break a rule before proving it: find the quantity it ignores.
3. EDF (inversions), Huffman (exchange + reduction), MST (cut property): the same pattern.
4. **Matroids** explain the pattern: greedy is optimal for every weight function exactly when the feasible sets form a matroid.
5. Not matroids: knapsack, matchings. They need DP (L07) or flow (L12).

NEXT WEEK · L09 (SAT 21 NOV)

MSTs and DSU: cut and cycle properties, Kruskal / Prim / Borůvka compared by density, and the amortized $O(\alpha(n))$ bound for union–find.

Read: CLRS 4e Ch. 19, 21; KT 4.5–4.6. **PS3** released next week.